

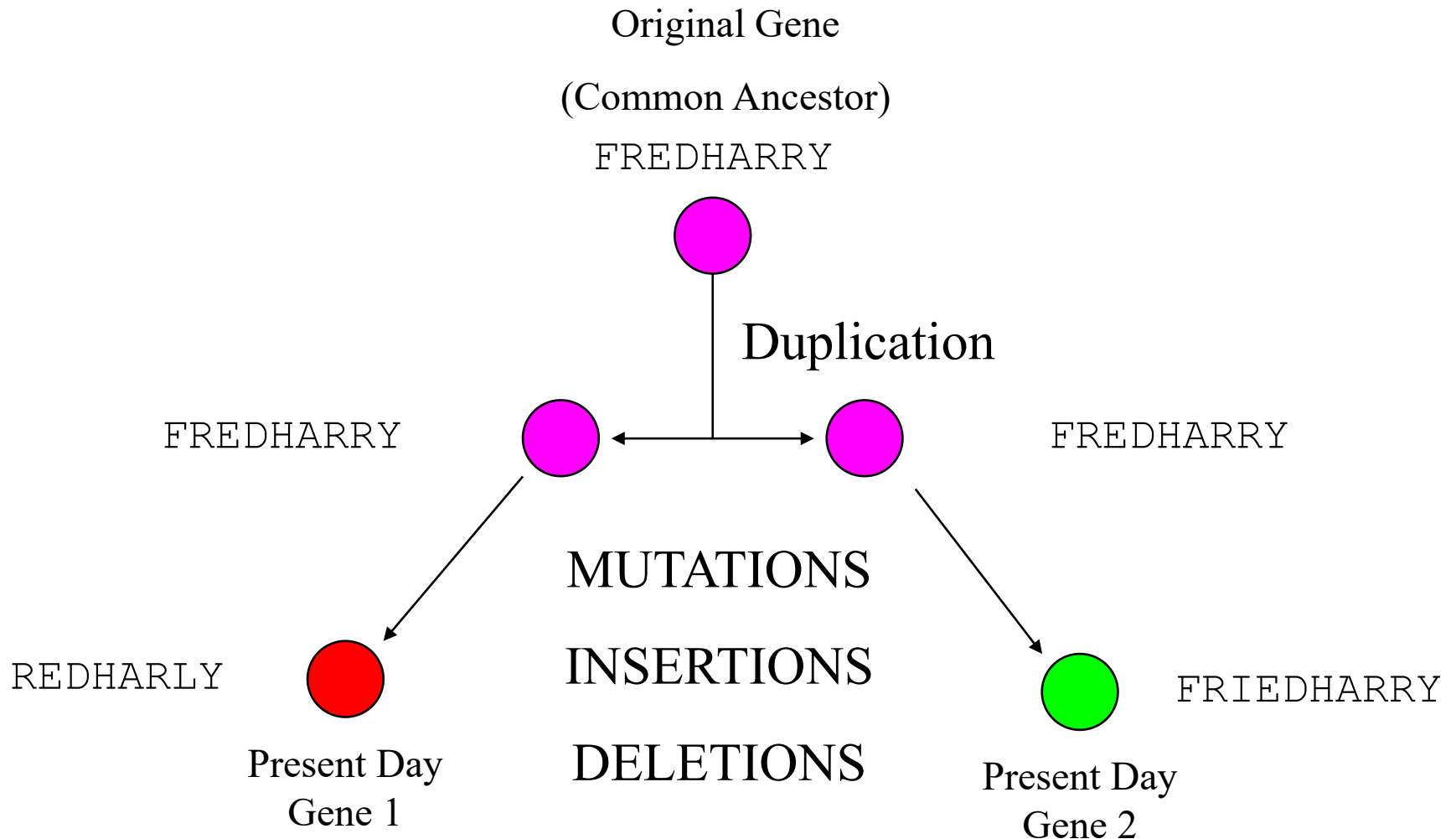
# **Protein Homology Detection**

## **Part 1**

Prof. David Jones  
d.t.jones@ucl.ac.uk

# Molecular Evolution :

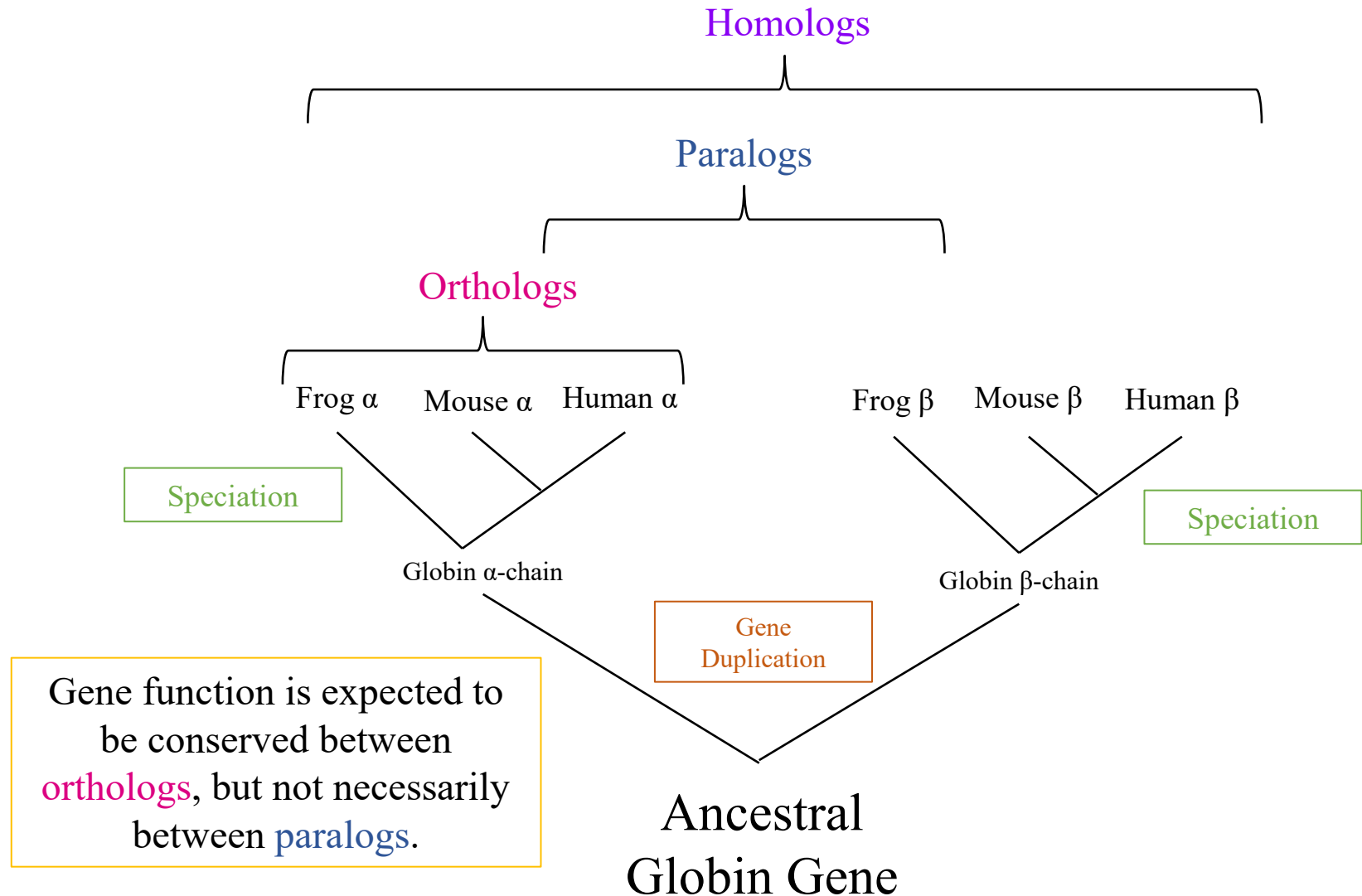
## Where do New Genes Come From?



# What is Homology?

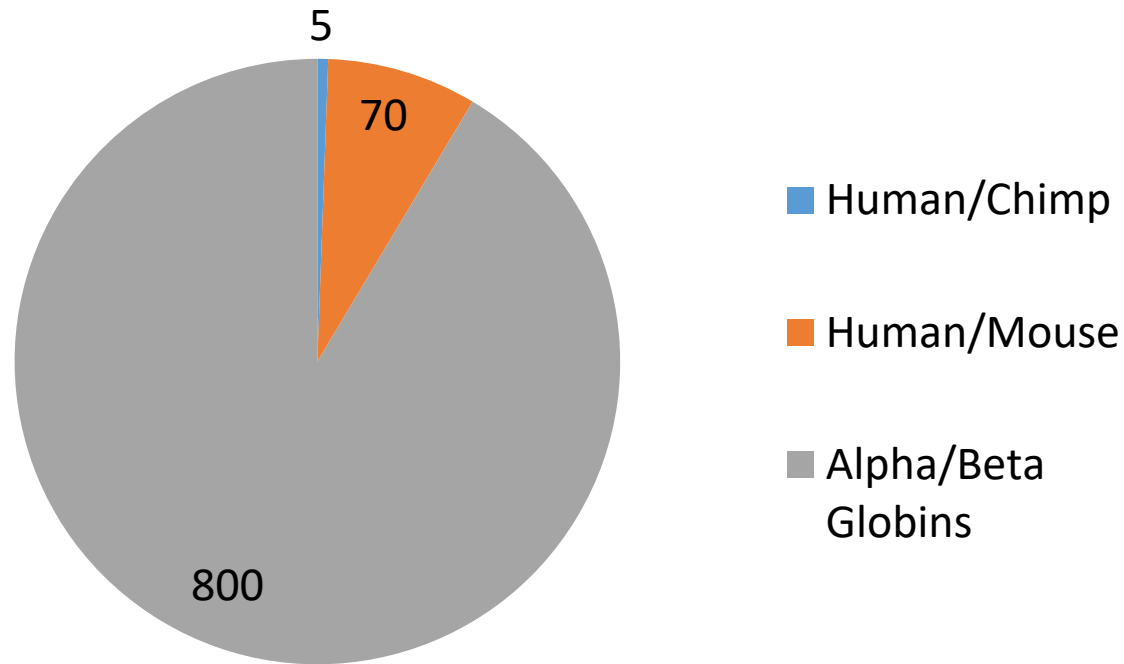
- Two genes or proteins are said to be **HOMOLOGOUS** if they have both evolved from a **COMMON ANCESTOR**
- Homology cannot be **MEASURED** i.e. it is impossible to be 50% **HOMOLOGOUS** – it is either **YES** or **NO**
- However, the **LIKELIHOOD** that two genes are related (i.e. are homologous) can be **ESTIMATED**

# Gene Duplication – Ortholog(ue)s vs Paralog(ue)s



# Clock of Globin Evolution

Time (in millions of years)



# Sequence Alignment

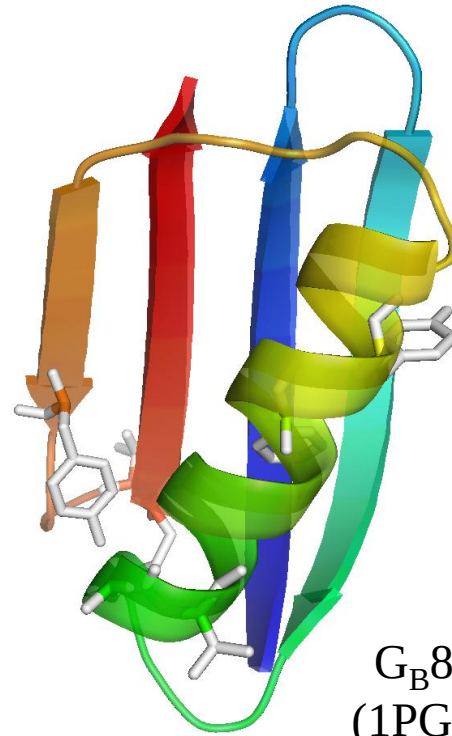
- > 30% amino acid identity between two protein sequences implies probable common structure and possibly common function
- However, this is just a “rule of thumb” and there are many exceptions

|      |                                                               |    |    |    |     |
|------|---------------------------------------------------------------|----|----|----|-----|
|      | 10                                                            | 20 | 30 | 40 | 50  |
| 1PLC | --IDVLLGADDGSLAFVPSEFSISPGEKIVFKNNAGFPHNIVFEDEDSIPS-GVDASKISM |    |    |    |     |
|      | : X:: : :: : ..... : : : :: ::::: :. ... : :                  |    |    |    |     |
| 1NIN | ETYYTVKLGSDDKGLLVFEPAKLTIKPGDTVEFLNNKVPPHNVVFDAALNPASADLAK-SL |    |    |    |     |
|      | 10                                                            | 20 | 30 | 40 | 50  |
| <br> |                                                               |    |    |    |     |
|      | 60                                                            | 70 | 80 | 90 |     |
| 1PLC | SEEDLLNAKGETFEVAL---SNKGEYSFYCSPHQGAGMVGKVTVN-                |    |    |    |     |
|      | :...:X . ... . :..... :.....                                  |    |    |    |     |
| 1NIN | SHKQLLMSPGQSTSTTFPADAPAGEYTFYCEPHRGAGMVGKITVAG                |    |    |    |     |
|      | 60                                                            | 70 | 80 | 90 | 100 |

# Paracelsus Proteins



G<sub>A</sub>88  
(2FS1)



G<sub>B</sub>88  
(1PGB)

|                   |                                                                                                                                                          |     |    |     |    |     |    |     |    |     |    |     |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----|----|-----|----|-----|----|-----|----|-----|----|-----|
|                   | 1                                                                                                                                                        | ... | 10 | ... | 20 | ... | 30 | ... | 40 | ... | 50 | ... |
| G <sub>A</sub> 88 | <div style="background-color: black; color: white; padding: 2px;">             TTYKLILNLKQAKEEAIKELVDAGIAEKYIKLIANAKTVEGVWTLKDEILTFTVTE           </div> |     |    |     |    |     |    |     |    |     |    |     |
| G <sub>B</sub> 88 | <div style="background-color: black; color: white; padding: 2px;">             TTYKLILNLKQAKEEAIKELVDAATAEKYFKLYANAKTVEGVWTYKDETKTFTVTE           </div> |     |    |     |    |     |    |     |    |     |    |     |

# Sequence Alignment Applications in Bioinformatics

- Sequence alignment algorithms are core to almost all areas of bioinformatics
- Mostly used to help infer evolutionary relationships between genes and their related proteins
  - Powerful clue in predicting function of gene/protein
  - Can produce an accurate 3-D model of protein if there is a known structure for the homologue found in the data bank
- Also useful for gene finding (see earlier lectures)
- Widely used as part of gene contig assembly in RNAseq/Next-gen sequencing genome analysis



# Is this alignment significant?

Query: 1 MIGQKTLYSFFSPSPARKRHAPSPEPAVQGTGVAGVPEESGDAAAIPAKKAPAGQEEP GT 60  
MIGQKTLYSFFSPSPARKRHAPSPEPAVQGTGVAGVPEESGDAAAIPAKKAPAGQEEP GT  
Sbjct: 1 MIGQKTLYSFFSPSPARKRHAPSPEPAVQGTGVAGVPEESGDAAAIPAKKAPAGQEEP GT 60

Query: 61 PPSSPLSAEQLDRIQRNKA AALLRLAARNVPVGFGESWKKHLSGEFGKPYFIKLMGFVAE 120  
PPSSPLSAEQLDRIQRNKA AALLRLAARNVPVGFGESWKKHLSGEFGKPYFIKLMGFVAE  
Sbjct: 61 PPSSPLSAEQLDRIQRNKA AALLRLAARNVPVGFGESWKKHLSGEFGKPYFIKLMGFVAE 120

Query: 121 ERKH YTVYPPPHQVFTWTQMCDIKDVKVILGQDPYHGPNQAHGLCF SVQRPVPPPPSLE 180  
ERKH YTVYPPPHQVFTWTQMCDIKDVKVILGQDPYHGPNQAHGLCF SVQRPVPPPPSLE  
Sbjct: 121 ERKH YTVYPPPHQVFTWTQMCDIKDVKVILGQDPYHGPNQAHGLCF SVQRPVPPPPSLE 180

Query: 181 NIYKELSTDIEDFVHPGHGDL SGWAKQGVLLLNAVLT VRAHQANSHKERGWEQFTDAVVS 240  
NIYKELSTDIEDFVHPGHGDL SGWAKQGVLLLNAVLT VRAHQANSHKERGWEQFTDAVVS  
Sbjct: 181 NIYKELSTDIEDFVHPGHGDL SGWAKQGVLLLNAVLT VRAHQANSHKERGWEQFTDAVVS 240

Query: 241 WLNQNSNGLV SLLWGSYAQKKGSAIDRKRHHVLQTAHPSPLSVYRGFFGCRHFSKTNELL 300  
WLNQNSNGLV LLWGSYAQKKGSAIDRKRHHVLQTAHPSPLSVYRGFFGCRHFSKTNELL  
Sbjct: 241 WLNQNSNGLV SLLWGSYAQKKGSAIDRKRHHVLQTAHPSPLSVYRGFFGCRHFSKTNELL 300

Query: 301 QKSGKKPIDWKEL 313  
QKSGKKPIDWKEL  
Sbjct: 301 QKSGKKPIDWKEL 313

# Is this?

Query: 5 QLFCVAEERSGHC AVVDGNFLYVWGGYVSIEDN---EVYLPNDEIWYTDIDSGLWRMHLM 61  
+L C AE RSGH AV DG ++VWGGY S + + YLP +E+W Y++++G W+

Sbjct: 25 ELACP AE-RSGH VAVSDGRHMFVWGGYKSNQVRGLYDFYLPREELWIYNMETGRWKKINT 83

Query: 62 EGELPASMSGSCGACINGKLYIFGGYDDKGYSNRLYFVNLRTDETYIWEKITDFEGQPP 121  
EG++P MSGSC C++ LY+FGG+ +G +N+ Y ++ R+ D WE+I D +G PP

Sbjct: 84 EGDVPPMSGSCAVCVDRVLYLFGGHHSRGNTNKFYMLDSRSTDRLQWERI-DCQGIPP 142

Query: 122 TPRDKLSCWVYKDRLIYFGGYGCRRHSELQDCFDVHDAS-WEEQIFWGWHNDVHIFDTKT 180  
+ +DKL WVYK++LI+FGGYG ++ F+ + S W GW++ VHI DT+T

Sbjct: 143 SSKDKLGVWVYK NKLIFFGGYGYLPEDKVLGTFEFDETSFWNSSHPRGWNDHVHILDTET 202

Query: 181 QTWFQPEIKGGVPPQPRAAHTCAVLGNKGYIFGGRVLQTRMNDLHYLNLDTWWSGRITI 240  
TW QP I G P PRAAH CA +GN+G++FGGR RMNDLHYLNLDTW W+ I

Sbjct: 203 FTWSQP-ITTGKAPSPRAAHACATVGNRGFVFGGRYRDARMNDLHYLNLDTWENELIP- 260

Query: 241 NGESPKHRSWHTLTPIADDKLFLCGGLSADNIPLSDGWIHNVTTCWKQLTHLPKTRPRL 300  
G P RSWH+LTP++ D LFL GG + D PLSD W + ++ N W Q H +PRL

Sbjct: 261 QGICPVGRSWHSLTPVSSDHLFLFGGF TTDKQPLSDAWTYCISKNEWIQFNHPYTEKPRL 320

Query: 301 WHTACLGKENEIMVFGGSKDDLALD-TGHCNDLLIFQTQPYSLLRSCLDCIGKNSIMLE 359  
WHTAC E E++VFGG ++LL H N++LIF QP SL+R L+ + ML

Sbjct: 321 WHTACASDEGEVIVFGGCANNLLVHHRAHSNEILIFSVQPKSLVRLSLEAVICFKEMLA 380

Query: 360 SQISLLPPKLLQQVLKKI 377  
+ + LP LL V ++

Sbjct: 381 NSWNC LPKHLLHSVNQRF 398

# What about this?

```

                190          200          210          220          230          240
2V0Z:C  SDPQH YEGNFHYINLIK TG V W Q I Q M K G V S V G S S T L L C E D G C L A L V D T G A S Y I S G S T S S I E
                ++ +WQ  +   +++G++  L E      AL+DTGA.      S+
QUERY  -----PQVTLWQRPIVTIKIGGQ--LKE-----ALLDTGADDTVLEEMSLP
                10          20          30

                250          260          270          280
2V0Z:C  -----KLMEALGA--KKRLFDYVVK--CNE-----GPTLPDISFHLGGKEYTLTSAD
                K++ ++G+  K R +D V    C++          GPT  +I   +G +  T   +
QUERY  GKWKPKMIGGIGGFIKVRQYDQVSIEICGHKAIGTVLIGPTPVNI---IGRNLLTQLGCT
                40          50          60          70          80          90

                290          300          310          320          330          340
2V0Z:C  YVFQESYSSKKLCTLAIHAMDIPPPTGPTWALGATFIRKIFYTEFDRRNNRIGFALAR
                F
QUERY  LNF-----
```

# Measures of Amino Acid Similarity

- Identity
- Similarity in properties (e.g. hydrophobicity or size)
- Similarity in Genetic Code (codons)
- Exchange propensities
  - Dayhoff Matrix
    - Well defined Markovian evolutionary model
    - Original calculated in 1978
    - Updated in 1992 by Jones et al.
  - BLOSUM Matrix
    - Based on ungapped “blocks” in alignments
    - No evolutionary model
    - Good for data bank searching

# Calculating Mutation Data Matrices

- Take aligned set of closely related proteins
- For each position in each alignment, count the number of substitutions (e.g. I  $\rightarrow$  V)
- Calculate relative frequency of occurrence for each amino acid
- Compute a log likelihood ratio with the reference state being a random co-occurrence (independent occurrence) model:

$$Mut(x \leftrightarrow y) = 10 \log_{10} \frac{f(x \leftrightarrow y)}{f(x)f(y)}$$

[illegible][illegible]

# The BLOSUM62 Matrix

|     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|---|--|
| Ala | 4   |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Arg | -1  | 5   |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Asn | -2  | 0   | 6   |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Asp | -2  | -2  | 1   | 6   |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Cys | 0   | -3  | -3  | -3  | 9   |     |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Gln | -1  | 1   | 0   | 0   | -3  | 5   |     |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Glu | -1  | 0   | 0   | 2   | -4  | 2   | 5   |     |     |     |     |     |     |     |     |     |     |     |     |   |  |
| Gly | 0   | -2  | 0   | -1  | -3  | -2  | -2  | 6   |     |     |     |     |     |     |     |     |     |     |     |   |  |
| His | -2  | 0   | 1   | -1  | -3  | 0   | 0   | -2  | 8   |     |     |     |     |     |     |     |     |     |     |   |  |
| Ile | -1  | -3  | -3  | -3  | -1  | -3  | -3  | -4  | -3  | 4   |     |     |     |     |     |     |     |     |     |   |  |
| Leu | -1  | -2  | -3  | -4  | -1  | -2  | -3  | -4  | -3  | 2   | 4   |     |     |     |     |     |     |     |     |   |  |
| Lys | -1  | 2   | 0   | -1  | -3  | 1   | 1   | -2  | -1  | -3  | -2  | 5   |     |     |     |     |     |     |     |   |  |
| Met | -1  | -1  | -2  | -3  | -1  | 0   | -2  | -3  | -2  | 1   | 2   | -1  | 5   |     |     |     |     |     |     |   |  |
| Phe | -2  | -3  | -3  | -3  | -2  | -3  | -3  | -3  | -1  | 0   | 0   | -3  | 0   | 6   |     |     |     |     |     |   |  |
| Pro | -1  | -2  | -2  | -1  | -3  | -1  | -1  | -2  | -2  | -3  | -3  | -1  | -2  | -4  | 7   |     |     |     |     |   |  |
| Ser | 1   | -1  | 1   | 0   | -1  | 0   | 0   | 0   | -1  | -2  | -2  | 0   | -1  | -2  | -1  | 4   |     |     |     |   |  |
| Thr | 0   | -1  | 0   | -1  | -1  | -1  | -1  | -2  | -2  | -1  | -1  | -1  | -1  | -2  | -1  | 1   | 5   |     |     |   |  |
| Trp | -3  | -3  | -4  | -4  | -2  | -2  | -3  | -2  | -2  | -3  | -2  | -3  | -1  | 1   | -4  | -3  | -2  | 11  |     |   |  |
| Tyr | -2  | -2  | -2  | -3  | -2  | -1  | -2  | -3  | 2   | -1  | -1  | -2  | -1  | 3   | -3  | -2  | -2  | 2   | 7   |   |  |
| Val | 0   | -3  | -3  | -3  | -1  | -2  | -2  | -3  | -3  | 3   | 1   | -2  | 1   | -1  | -2  | -2  | 0   | -3  | -1  | 4 |  |
| Ala | Arg | Asn | Asp | Cys | Gln | Glu | Gly | His | Ile | Leu | Lys | Met | Phe | Pro | Ser | Thr | Trp | Tyr | Val |   |  |

The BLOSUM matrices are now the standard matrices for sequence alignment. They are based on comparing present-day related sequences (as described previously), unlike the Dayhoff matrices, which are based on comparing present-day sequences to inferred common ancestor sequences.

# Sequence Comparison by Dynamic Programming

|      |                                                                       |    |    |    |     |
|------|-----------------------------------------------------------------------|----|----|----|-----|
|      | 10                                                                    | 20 | 30 | 40 | 50  |
| 1PLC | --IDVLLGADDGSLAFVPSEFSISPGEKIVFKNNAGFPHNIVFDEDSIPS-GVDASKISM          |    |    |    |     |
|      | : X:: : :: : . . . . . : : :: : . . . . . : . . . . . : . . . . .     |    |    |    |     |
| 1NIN | ETYTVKLGSDKGLLVFEPAKLTIKPGDTVEFLNNKVPPHNVVFDAALNPAKSADLAK-SL          |    |    |    |     |
|      | 10                                                                    | 20 | 30 | 40 | 50  |
|      | 60                                                                    | 70 | 80 | 90 |     |
| 1PLC | SEEDLLNAKGETFEVAL---SNKGEYSFYCSPHQGAGMVGKVTVN-                        |    |    |    |     |
|      | : . . . : X . . . . . : . . . . . : . . . . . : . . . . . : . . . . . |    |    |    |     |
| 1NIN | SHKQLLMSPGQSTSTTFPADAPAGEYTFYCEPHRGAGMVGKITVAG                        |    |    |    |     |
|      | 60                                                                    | 70 | 80 | 90 | 100 |



# Dynamic programming algorithm for global sequence comparison

- When is it useful?
  - To compare similarity in two sequences of possibly different lengths (i.e. allowing gaps)
  - But where the sequences are presumed to be related over their entire lengths
- Introduced for biological sequences by S. B. Needleman & C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *J. Mol. Biol.* 48:443-453 (1970)

# Some historical trivia

- The NW algorithm described in 1970 didn't include a gap penalty term but briefly described how it could be added
- In fact the original NW algorithm as described was  $O(n^3)$ , though a recurrence formula for an algorithm  $O(n^2)$  was briefly mentioned in the discussion section
- Later work by Sankoff (1972) produced the modern form of the “NW algorithm” which is  $O(n^2)$
- In fact an algorithm for speech-warping by Vintsyuk (1968) is very similar to the NW/Sankoff algorithm
- Today we call all  $O(n^2)$  dynamic programming global sequence alignment algorithms “Needleman-Wunsch” algorithms

# Gap penalties

- Linear Gap penalty

- cost of gap (length  $g$ ) depends linearly on *gap-open penalty*
  - $f(g) = -gd$

- Affine gap penalty

- cost of gap depends on an initial gap-open penalty and a subsequent *gap-extension* penalty
- based on the fact that a single biological mutational event can insert or delete more than one residue
  - $f(g) = -d - (g-1)e$

*currently there is no widely accepted theory for calculating gap costs  
- generally guided by empirical tests or trial and error.*

# Building $M(i,j)$

- Dynamic programming algorithm reduces time complexity to  $O(nm)$
- $M(i,j)$  built recursively by choosing the best scoring alignment from three prior scenarios by:
  - “(mis)match” where  $x_i$  aligns with  $y_j$
  - “insertion” in  $x$  or “deletion” in  $y$  so that  $x_i$  aligns with a gap
  - “deletion” in  $x$  or “insertion” in  $y$  so that  $y_j$  aligns with a gap

# NW GLOBAL ALIGNMENT ALGORITHM with LINEAR GAP PENALTY

Initialisation :

$$M(0,0) = 0,$$

$$M(i,0) = -id, \quad 1 \leq i \leq n,$$

$$M(0,j) = -jd, \quad 1 \leq j \leq m,$$

Recursion formula :

$$M(i,j) = \max \begin{cases} M(i-1,j-1) + \text{score}(x_i, y_j) \\ M(i,j-1) - d \\ M(i-1,j) - d \end{cases} \quad d : \text{gap open penalty}$$

$\text{score}(x_i, y_j)$  : score of aligning element  $i$  of sequence  $x$  with element  $j$  of sequence  $y$ ,  
generally obtained from a scoring (or substitution) matrix e.g. BLOSUM50

Note :  $M$  is an  $(n+1)$  by  $(m+1)$  matrix.

$x$  :

|       |       |    |    |    |       |    |       |
|-------|-------|----|----|----|-------|----|-------|
| $x_1$ | $x_2$ | .. | .. | .. | $x_i$ | .. | $x_n$ |
|-------|-------|----|----|----|-------|----|-------|

$y$  :

|       |       |    |    |    |    |    |    |    |       |    |       |
|-------|-------|----|----|----|----|----|----|----|-------|----|-------|
| $y_1$ | $y_2$ | .. | .. | .. | .. | .. | .. | .. | $y_j$ | .. | $y_m$ |
|-------|-------|----|----|----|----|----|----|----|-------|----|-------|

# Global Alignment Example

|   |    | C  | A  | L  | M  | E  | R  |
|---|----|----|----|----|----|----|----|
|   | 0  | -1 | -2 | -3 | -4 | -5 | -6 |
| C | -1 | 1  | 0  | -1 | -2 | -3 | -4 |
| A | -2 | 0  | 2  | 1  | 0  | -1 | -2 |
| M | -3 | -1 | 1  | 1  | 2  | 1  | 0  |
| E | -4 | -2 | 0  | 0  | 1  | 3  | 2  |
| A | -5 | -3 | -1 | -1 | 0  | 2  | 2  |
| R | -6 | -4 | -2 | -2 | -1 | 1  | 3  |

The traceback path is shown in red and gives this optimal alignment:

```

C  A  L  M  E  -  R
C  A  -  M  E  A  R
Score = 3

```

Here we score matches with a score of 1 and mismatches with a score of -1. Gap penalty is set 1.

# The Gotoh Algorithm

- Gotoh (1982) proposed a more useful algorithm for NW alignments which can handle affine gap penalties, still in order  $nm$  time.
- In addition to the optimum alignment score, we also need to track the best score given that  $x_i$  is aligned to a gap,  $N(i, j)$ , and the best score given that  $y_j$  is in an insertion with respect to  $x$ ,  $O(i, j)$  so that we can separate the processes of opening and extending gaps.
- The recursion in this case is as follows:

$$M(i, j) = \max\{M(i-1, j-1), N(i-1, j), O(i, j-1)\} + \text{score}(x_i, y_j),$$

$$N(i, j) = \max\{M(i-1, j-1) - d, N(i-1, j) - g\},$$

$$O(i, j) = \max\{M(i-1, j-1) - d, O(i, j-1) - g\}$$

where  $d$  is the cost of opening a gap, and  $g$  the cost of extending one that has already been opened.

# Local Sequence Alignment

- When is it useful?
  - to compare subsequences of two sequences  $x$  and  $y$
  - to identify a known domain/sequence motif in a new protein
  - comparing a long DNA sequence with a short one (e.g. a genome vs a single gene)
  - to identify local similarities (e.g. common domains) in two highly diverged sequences

SMITH-WATERMAN ALGORITHM (1981)



# SW LOCAL ALIGNMENT ALGORITHM with LINEAR GAP PENALTY

Initialization:

$$M(0,0) = 0,$$

$$M(i,0) = 0 [= \max(0, -id)], \quad 1 \leq i \leq n,$$

$$M(0,j) = 0 [= \max(0, -jd)], \quad 1 \leq j \leq m,$$

Recursion formula :

$$M(i,j) = \max \begin{cases} M(i-1,j-1) + \text{score}(x_i, y_j) \\ M(i,j-1) - d \\ M(i-1,j) - d \\ 0 \end{cases} \quad d : \text{gap open penalty}$$

$\text{score}(x_i, y_j)$ : score of aligning element  $i$  of sequence  $x$  with element  $j$  of sequence  $y$ ,  
generally obtained from a scoring (or substitution) matrix e.g. BLOSUM50

Note:  $M$  will be an  $(n+1) \times (m+1)$  matrix.  $i$  labels rows,  $j$  labels columns

$x :$

|       |       |    |    |    |       |    |       |
|-------|-------|----|----|----|-------|----|-------|
| $x_1$ | $x_2$ | .. | .. | .. | $x_i$ | .. | $x_n$ |
|-------|-------|----|----|----|-------|----|-------|

$y :$

|       |       |    |    |    |    |    |    |    |       |    |       |
|-------|-------|----|----|----|----|----|----|----|-------|----|-------|
| $y_1$ | $y_2$ | .. | .. | .. | .. | .. | .. | .. | $y_j$ | .. | $y_m$ |
|-------|-------|----|----|----|----|----|----|----|-------|----|-------|

# Smith-Waterman Alignment Example

|   |   | H | P | E | A | A | W | G | H | E | V |
|---|---|---|---|---|---|---|---|---|---|---|---|
| P | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| W | 0 | 0 | 0 | 0 | 0 | 1 | 4 | 2 | 0 | 0 | 0 |
| H | 0 | 2 | 0 | 0 | 0 | 0 | 2 | 3 | 4 | 2 | 0 |
| E | 0 | 0 | 1 | 2 | 0 | 0 | 0 | 1 | 2 | 6 | 4 |
| A | 0 | 0 | 0 | 0 | 4 | 2 | 0 | 0 | 0 | 4 | 5 |
| E | 0 | 0 | 0 | 2 | 2 | 3 | 1 | 0 | 0 | 2 | 3 |
| W | 0 | 0 | 0 | 0 | 1 | 1 | 5 | 3 | 1 | 0 | 1 |
| Y | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 4 | 2 | 0 | 0 |

AWGHE

AW-HE

Score = 6

Match = 2

Mismatch = -1

Gap = 2

# The Smith-Waterman Algorithm

- The SW algorithm guarantees to find the optimum scoring local alignment between two sequences
- The optimum local alignment could be identical to the optimum global alignment
- There could be other significant local alignments, but SW will just return the highest scoring one
- We can find other significant alignments by masking out the initial SW alignment and re-running the algorithm
- Note that SW will only produce a local alignment under certain conditions i.e. with certain score and gap penalty choices e.g. a score matrix without any negative scores is unlikely to result in a local alignment
- Necessary conditions for local alignment behaviour are hard to predetermine, but as a rule of thumb, for aligned random sequences, the expectation value of the alignment score should be negative for local behaviour to result.

# Efficient database searching methods

- Dynamic programming requires order  $N^2L$  computations (where  $N$  is size of the query sequence and  $L$  is the size of the database)
- Given size of databases, more efficient methods needed

# Approximate searching: “Hit and extend” sequence comparison

- Problem: Too many calculations “wasted” by comparing regions that have nothing in common
- Initial insight: Regions that are *similar* between two sequences are likely to share short stretches that are *identical*
- Basic method: Look for similar regions only near short stretches that match *exactly*

# “Hit and extend” sequence searching

- We define a **word** size that is the minimum number of exact “letter” matches that must occur before we do any further comparison or alignment
- How do we find all of the occurrences of matching words between a sequence and a database?
  - Could scan sequence a word at a time, but this is order  $L$  (size of database)

# Word searching

- Solution: Use a precomputed table that lists where in the database each possible word occurs
  - *Generation* of the table is of order  $L$  (size of database) but *use* of the table is of order  $N$  (size of query sequence)

# FASTA

- Widely used for searching sequence data banks until the advent of BLAST
- Inputs
  - k (word or ktuple) size
  - similarity matrix
- Compares query sequence pairwise with each sequence in the database



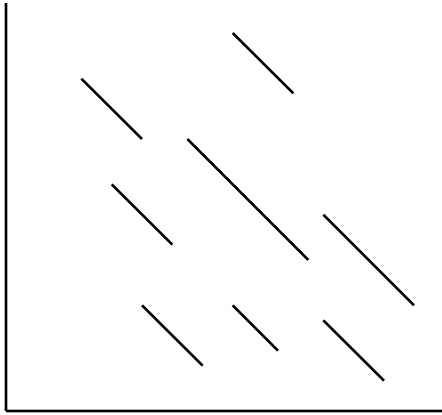
# FASTA method

1. Find **diagonals** (paired pieces from each sequence without gaps) that have the highest density of common words
2. Rescore these using a scoring (similarity) matrix and trim ends that do not contribute to the highest score
  - Result: partial alignments *without* gaps
  - Reported as the “init1” score

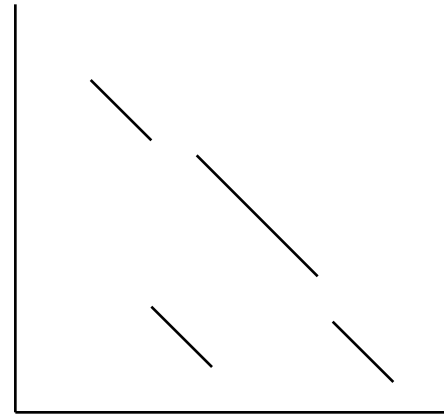
# FASTA method

3. Join regions together, including penalties for gaps
  - Result: *unoptimized* alignment with gaps
  - Reported as the “initn” score
4. Use dynamic programming in a band 32 residues wide around the best “initn” score
  - Result: optimized alignment with gaps
  - Reported as the “opt” score

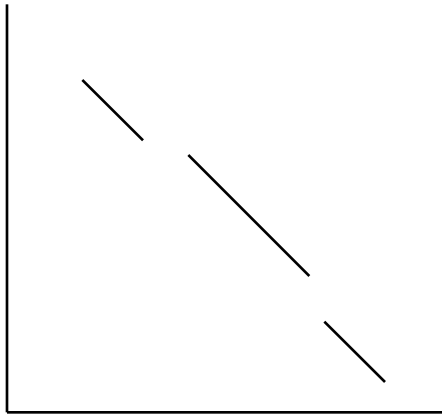
# FASTA Processing Steps



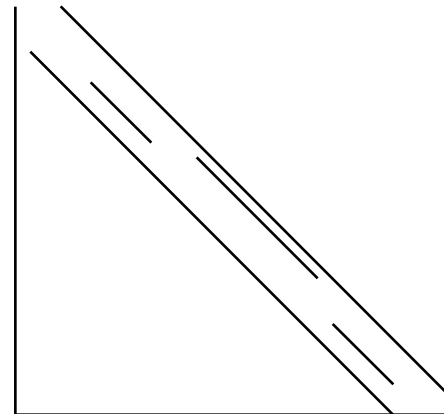
a) Find diagonals with many common tuples



b) Rescore using PAM matrix and keep best scoring diagonals



c) Eliminate overlapping diagonals with lower scores



d) Align using dynamic programming (in a band around best diagonals)

# Comments on FASTA

- Larger ktuple increases speed since fewer “hits” are found but it also decreases sensitivity for finding similar but not identical sequences since exact matches of this length are required

# Limitations of FASTA

- FASTA can miss significant similarity since
  - For proteins, similar sequences do not have to share identical residues
    - **Asp-Lys-Val** is quite similar to **Glu-Arg-Ile** yet it is missed even with ktuple size of 1 since none of the amino acids matches
    - **Gly-Asp-Gly-Lys-Gly** is quite similar to **Gly-Glu-Gly-Arg-Gly** but there is no match with ktuple size of 2 (no common words of length 2)

# BLAST (Basic Local Alignment Search Tool)

- Goal: find sequences from database similar to query sequence
- Previous tools use either
  - direct, theoretically sound but computationally slow approach to examine all possible alignments of query with database (dynamic programming)
  - indirect, heuristic but computationally fast approach to find similar sequences by first finding identical stretches (FASTP, FASTA)

# BLAST (Basic Local Alignment Search Tool)

- BLAST combines best of both by using theoretically sound method which searches for similar sequences directly but is computationally fast
- Current version uses words (3-tuples by default) to identify significant diagonals and then a final score computed again using banded Smith Waterman and a BLOSUM scoring matrix (default BLOSUM62). Optionally, a full SW alignment can be carried out on top-hits.
- Similar to FASTA, but builds an index of the whole data bank rather than each sequence i.e. BLAST treats data bank like one long sequence. Also includes more sophisticated heuristics after the initial word matching to eliminate matched which cannot be significant.
- References
  - **S. F. Altschul, W. Gish, W. Miller, E. W. Myers and D. J. Lipman.** Basic Local Alignment Search Tool. *J. Mol. Biol.* 215:403-410 (1990)
  - **Altschul, S.F., Madden, T.L., Schäffer, A.A., Zhang, J., Zhang, Z., Miller, W. & Lipman, D.J.** (1997) "Gapped BLAST and PSI-BLAST: a new generation of protein database search programs." *Nucleic Acids Res.* 25:3389-3402.

Time to search UniRef50 DB ( $\approx 12,000,000$  protein sequences) with 153 residue query sequence:

| Method                  | Time (seconds) |
|-------------------------|----------------|
| Standard Smith-Waterman | 2760           |
| FASTA (ktup=2)          | 132            |
| SIMD (SSSE3) S-W        | 55             |
| BLAST                   | 32             |

To think about: If the query sequence was much longer, then FASTA (and BLAST) would both beat SIMD S-W. Why?



```

opt      E ()
< 20    241      0:==
22       0       0:          one = represents 168 library sequences
24       0       0:
26       8       2:*
28      34      24:*
30     251     143:*=
32     670     554:===*
34    1591    1502:=====*=
36    3034    3085:=====*
38    4867    5098:===== *
40    6727    7112:===== *
42    8654    8693:===== *
44    9954    9589:=====*=
46   10027    9767:=====*=
48    9809    9351:=====*=
50    8729    8533:=====*=
52    7524    7502:===== *
54    6292    6408:===== *
56    5176    5352:===== *
58    4263    4394:===== *
60    3440    3560:===== *
62    2816    2854:===== *
64    2273    2270:===== *
66    1678    1794:===== *
68    1273    1411:===== *
70    1059    1106:===== *
72     818     864:===== *
74     588     674:===== *
76     477     524:===== *
78     374     408:===== *
80     287     316:===== *
82     222     242:===== *
84     164     192:===== *
86     132     148:===== *
88     107     115:===== *          inset = represents 15 library sequences
90      81      89:===== *
92      55      69:===== *
94      40      53:===== *
96      27      41:===== *
98      16      32:===== *
100     12      25:===== *
102      6      19:===== *
104     11      15:===== *
106      9       11:===== *
108      6       9:===== *
110      4       7:===== *
112      4       5:===== *
114      4       4:===== *
116      4       3:===== *
118      4       2:===== *
>120    716      2:===== *

```

## FASTA Output

# FASTA Output

The best scores are:

|                                                   |        | opt  | bits | E(104558) |
|---------------------------------------------------|--------|------|------|-----------|
| sp P02185 MYG_PHYCA MYOGLOBIN.                    | ( 153) | 1003 | 230  | 1.4e-60   |
| sp P02184 MYG_KOGSI MYOGLOBIN.                    | ( 153) | 979  | 225  | 5.5e-59   |
| sp P02177 MYG_ESCGI MYOGLOBIN.                    | ( 153) | 943  | 217  | 1.4e-56   |
| sp P02178 MYG_MEGNO MYOGLOBIN.                    | ( 153) | 935  | 215  | 4.7e-56   |
| sp P02181 MYG_INIGE MYOGLOBIN.                    | ( 153) | 934  | 215  | 5.5e-56   |
| sp P02174 MYG_GLOME MYOGLOBIN.                    | ( 153) | 931  | 214  | 8.6e-56   |
| sp P02173 MYG_ORCOR MYOGLOBIN.                    | ( 153) | 927  | 213  | 1.6e-55   |
| sp P02172 MYG_TURTR MYOGLOBIN.                    | ( 153) | 926  | 213  | 1.9e-55   |
| .                                                 |        |      |      |           |
| .                                                 |        |      |      |           |
| sp P11749 HBB_THUTH HEMOGLOBIN BETA CHAIN.        | ( 146) | 104  | 31   | 1.1       |
| sp P02135 HBB_RANCA HEMOGLOBIN BETA CHAIN.        | ( 140) | 103  | 31   | 1.2       |
| sp P02013 HBA2_XENLA HEMOGLOBIN ALPHA-2 CHAIN (MI | ( 141) | 103  | 31   | 1.2       |
| sp P31331 GLB_NASMU GLOBIN (MYOGLOBIN) .          | ( 147) | 103  | 31   | 1.2       |
| sp Q23762 GLBK_CHITH GLOBIN CTT-VIIB-10 PRECURSOR | ( 161) | 103  | 31   | 1.3       |
| sp Q23761 GLBZ_CHITH GLOBIN CTT-Z PRECURSOR (HBZ) | ( 163) | 103  | 31   | 1.3       |
| sp P18966 GLBI_CHITP GLOBIN CTT-VIIB-8 PRECURSOR. | ( 161) | 102  | 31   | 1.5       |
| sp P12550 GLBH_CHITH GLOBIN CTT-VIIB-7 PRECURSOR. | ( 162) | 102  | 31   | 1.6       |
| sp P29245 GLBZ_CHITP GLOBIN CTT-Z PRECURSOR (HBZ) | ( 163) | 102  | 31   | 1.6       |
| sp P18979 HBA1_UROHA HEMOGLOBIN ALPHA-1 CHAIN (FR | ( 90)  | 99   | 30   | 1.7       |
| sp P14395 GLBA_ANATR GLOBIN I ALPHA CHAIN.        | ( 150) | 101  | 30   | 1.7       |
| sp O13078 HBB_MERMR HEMOGLOBIN BETA CHAIN.        | ( 146) | 100  | 30   | 2         |

# Testing the Significance of Alignments

- Can we prove that two sequences are related?
  - NO!
- But we can estimate the **probability** that they are related

# Testing the Significance of Alignments

- We can ask the following question...
- What is the chance of finding two completely random sequences which have a match score greater than or equal to the given score?

# Testing the Significance of Alignments

- First approach used was to shuffle sequences (say 100 times) and calculate the probability of a certain score from the shuffled sequence scores.
- For the 100 random comparisons we calculate the mean and standard deviation.

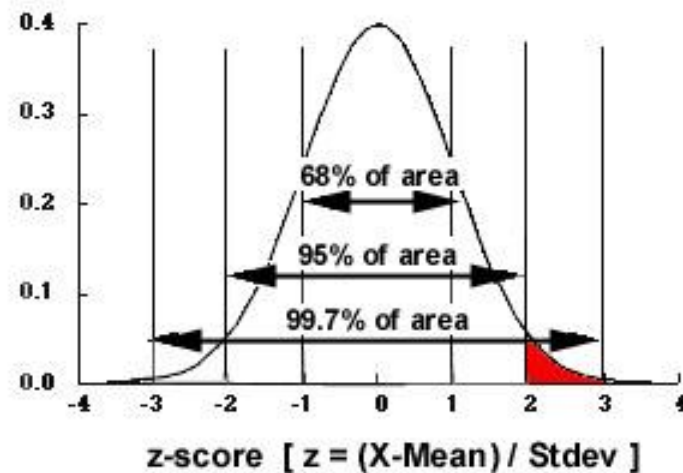
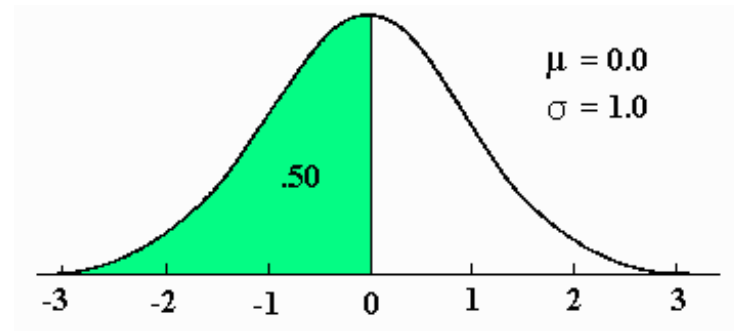
# Statistical Significance

- Can calculate a Z-score from score (S) using mean of all match scores plus the standard deviation ( $\sigma$ ) as follows:

$$Z = \frac{\textit{Score} - \bar{S}}{\sigma}$$

- Significance can be assessed according to the normal distribution

# The Normal Curve (Gaussian)

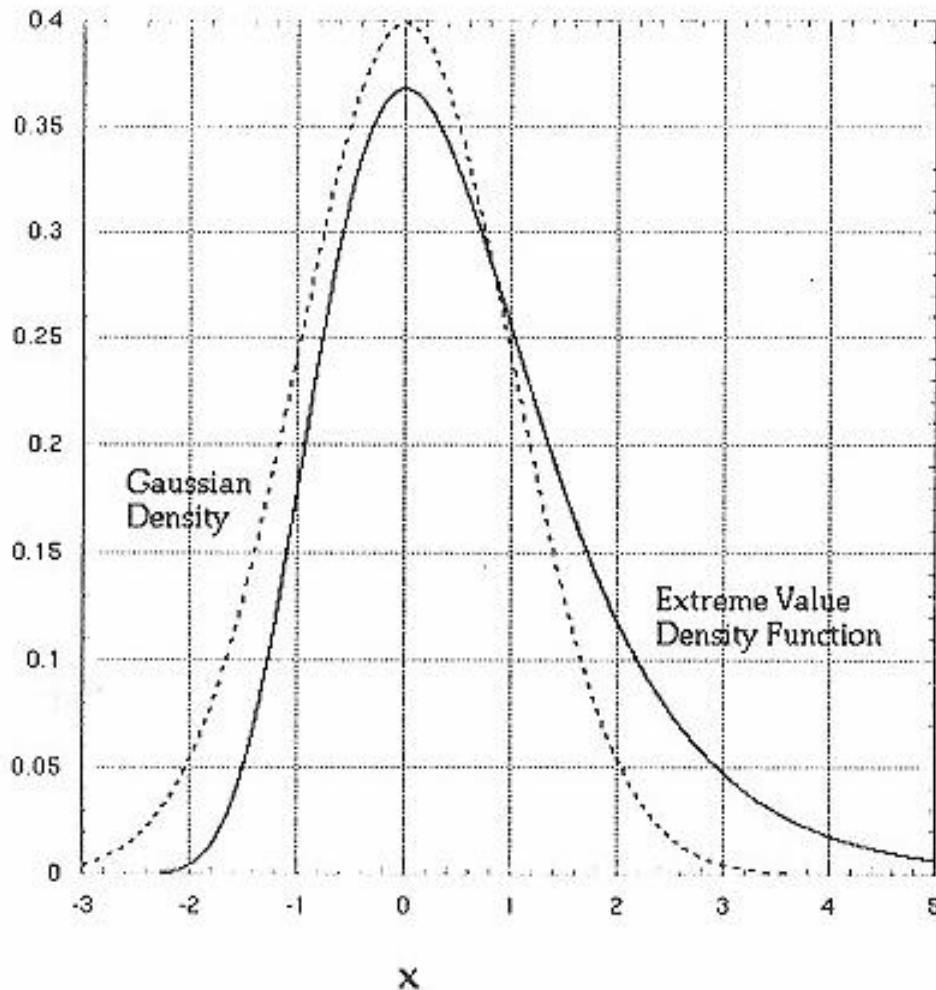


# Are random sequence alignment scores normally distributed?

- Although the distribution of scores from alignments of random sequences “look” normal they clearly are skewed towards larger values
- This is typical of “extreme” measurements: e.g. tallest person in a group
- The “Extreme Value Distribution” is a much better model for alignment scores



# The Gumbel Extreme Value Distribution



*The CDF of the Gumbel EVD has the form:*

$$P(S \geq x) = 1 - \exp(-e^{-\lambda(x-\mu)})$$

For ungapped alignments of two random sequences of length  $m$  and  $n$ :

$$\mu = \log Kmn / \lambda$$

The probability of an alignment score  $S$  being greater than or equal to  $x$  for  $x \gg 0$  is therefore:

$$P(S \geq x) = 1 - \exp(-Kmn e^{-\lambda x})$$

$K$  and  $\lambda$  can be calculated from the data in the scoring matrix used and from the relative frequencies of the amino acids (or nucleotides) in the data bank.

# Two ways to calculate the significance of scores

- Direct Curve-fitting
  - Slow but simple and effective
  - Need all scores before we can estimate  $K$  and  $\lambda$
  - Can be applied to arbitrary score matrices and gap penalties
- Simplified approach used in BLAST
  - For average amino acid composition sequences and a given score matrix and gap penalty,  $K$  and  $\lambda$  are more or less constant and can be precalculated based on random sequence shuffling simulations (using above direct curve-fitting approach)
  - Can estimate  $p$ -value for any match without knowing other match scores, but only if  $K$  and  $\lambda$  have already been computed. If you change matrix or gap penalty a new  $K$  and  $\lambda$  will be required!

# $E$ -values and $p$ -values in BLAST

$E$ -value: The expected *number* of matches with score of at least  $S$  (e.g. alignment bit score)

$$E = Kmn e^{-\lambda S}$$

Where  $m, n$  are sequence lengths

$K, \lambda$  are EVD background distribution parameters

$p$ -value: The *probability* of having at least one unrelated match with score of at least  $S$

$$p = 1 - e^{-E}$$